network.py

```python
from os import _exit
import websockets
import threading
import asyncio
from tkinter import messagebox
import queue
import json

class NetworkHandler:
    def __init__(self, master, websocketURL: str):
        self.wsURL: str = websocketURL
        self.master = master

        self.running: bool = False

        self.queueIncoming = queue.Queue()
        self.queueOutgoing = queue.Queue()

        self.master.after(100, self.processIncoming)

    def sendAction(self, action: dict):
        if self.running:
            self.queueOutgoing.put(json.dumps(action))

    async def receiver(self, websocket: websockets.ClientConnection):
        try:
            while True:
                message = await websocket.recv()
                self.queueIncoming.put(message)

        except websockets.ConnectionClosed:
            print("Disconnecting")

    async def sender(self, websocket: websockets.ClientConnection):
        try:
            while True:
                message = await asyncio.to_thread(self.queueOutgoing.get)
                await websocket.send(message)

        except websockets.ConnectionClosed:
            print("Disconnecting")

    async def wsHander(self):
        for attempt in range(10):
            try:
                async with websockets.connect(self.wsURL) as websocket:
                    self.websocket = websocket
```

```
48
49                      await asyncio.gather(
50                          self.sender(websocket),
51                          self.receiver(websocket)
52                      )
53                      return
54
55              except OSError as error:
56                  print(f"Retry {attempt}: {error}")
57                  await asyncio.sleep(3)
58
59          self.master.after(0, lambda: messagebox.showerror(
60              "Connection Error",
61              f"Failed to connect after retries:\n{self.wsURL}"
62          ))
63
64
65      def processIncoming(self):
66          while not self.queueIncoming.empty():
67              data: dict = json.loads(self.queueIncoming.get())
68              self.master.runAction(data)
69
70          self.master.after(100, self.processIncoming)
71
72      def runWsHandler(self):
73          self.asyncioLoop = asyncio.new_event_loop()
74          asyncio.set_event_loop(self.asyncioLoop)
75
76          self.asyncioLoop.run_until_complete(
77              self.wsHander()
78          )
79
80      def stop(self):
81          if self.running:
82              if hasattr(self, "websocket"):
83                  self.asyncioLoop.call_soon_threadsafe(
84                      lambda: asyncio.create_task(
85                          self.websocket.close(code = 1000, reason = "Application
    Closed")
86                      )
87                  )
88
89              _exit(0)
90
91      def run(self):
92          self.running = True
93
94          self.wsThread = threading.Thread(
95              target = self.runWsHandler,
```

```
            daemon = True
        )

        self.wsThread.start()

```